

SWARM PASTOR Video Game Development Report

David López Escoriza

1. Introduction

Swarm Pastor is a real-time 2D game that explores indirect control and emergent behavior through swarm-based artificial intelligence. Instead of commanding units directly, the player influences a group of autonomous agents by controlling a single drone, acting as a shepherd rather than a traditional leader.

The core of the game is built around swarm intelligence principles inspired by natural flocking behavior. Each agent follows simple local rules, yet complex global patterns emerge from their interactions. The gameplay challenge arises from protecting the swarm against predators in a hostile environment, where success depends on positioning, timing, and spatial awareness rather than predefined strategies.



1.1 Motivation

The motivation behind *Swarm Pastor* arises from a strong interest in complex systems and emergent behavior in interactive environments. This project explores the idea of indirect control, where simple local rules give rise to complex global behaviors.

On a personal level, the project serves as an exploration of artificial intelligence techniques that do not rely on heavy computation or machine learning, but instead on lightweight, interpretable rules inspired by natural phenomena such as flocking and collective motion. This makes the game not only interactive but also a form of playable simulation.

1.2 Technology Used

To achieve a balance between real-time simulation, emergent artificial intelligence, and modular software design, the project relies on the following technological stack:

Programming Language (Python 3):

Python facilitates a clean separation between game logic and rendering concerns.

Framework (Pygame):

Pygame is used as the low-level framework responsible for window management, input handling, timing control, and 2D rendering. [4]

Entity–System Architecture:

The internal structure of the game follows an entity–system-oriented design. Game objects are represented as entities with minimal internal state, while behavior is implemented through independent systems (e.g., swarm behavior, predator logic, player control).

Event-Oriented Gameplay Flow:

Game progression elements such as enemy spawning, power usage, and game-over conditions are triggered through well-defined events. This decouples gameplay logic from presentation and facilitates future extensions such as new enemy types, abilities, or game modes.

1.3 State of the Art and Analysis of Similar Titles

Swarm Pastor is positioned at the intersection of two well-established design paradigms in interactive systems: swarm-based artificial intelligence and indirect control gameplay. Rather than focusing on explicit command structures or scripted behaviors, the project draws inspiration from systems where complexity emerges from simple local rules and player influence is mediated through the environment.

This section analyzes the main mechanical and conceptual influences that informed the design of the game, as well as adjacent works that explore similar ideas from different perspectives.

1.3.1 Mechanical Foundations: Boids and Swarm Intelligence

The primary mechanical influence for *Swarm Pastor* is the Boids model, originally proposed by Craig Reynolds to simulate flocking behavior through a small set of local rules such as cohesion, separation, and alignment [1]. This model has become a foundational reference for swarm intelligence in animation, robotics, and game development.

While many games employ swarm techniques primarily for visual effect, *Swarm Pastor* integrates swarm behavior as a core gameplay mechanic. The survival of the group depends directly on maintaining swarm cohesion, making emergent behavior a functional requirement rather than a cosmetic feature.

1.3.2 Indirect Control and Escort-Based Gameplay

A second major influence comes from games and simulations that emphasize indirect player control, where the player shapes outcomes through positioning and environmental interaction rather than issuing explicit commands [2].

Traditional real-time strategy games rely on deterministic unit control and predefined responses. In contrast, *Swarm Pastor* deliberately removes direct control over individual agents. The player interacts with the system through a single drone whose presence alters swarm behavior probabilistically rather than deterministically.

1.3.3 Emergent Systems and Systemic Gameplay

From a broader design perspective, *Swarm Pastor* aligns with games that prioritize systemic gameplay, where mechanics interact dynamically to produce unscripted outcomes [3]. In such systems, player agency arises from understanding the rules governing the simulation rather than from scripted progression.

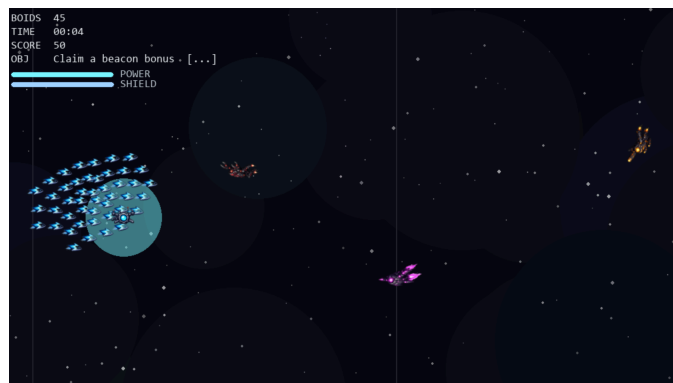
The continuous pressure applied by predators introduces instability into the system, forcing the player to adapt to evolving conditions. This reflects a design philosophy in which tension is generated by systemic interactions rather than authored events.

1.4 Game Proposal

Swarm Pastor is a real-time survival game built around **indirect control, spatial pressure, and emergent swarm behavior**. The player influences a fragile group of autonomous agents by controlling a single drone, whose position shapes swarm movement without direct commands.

Gameplay revolves around maintaining swarm cohesion under constant threat. Predators continuously disrupt the group, forcing the player to balance protection, repositioning, and regrouping. Individual agents are intentionally fragile, and their loss reduces overall swarm stability, leading to a **cascading failure dynamic** where early mistakes amplify long-term difficulty.

Progression emerges naturally from system interaction rather than scripted events. Difficulty escalates through increased environmental and predatory pressure, and the game ends when the swarm collapses entirely, framing failure as a systemic outcome rather than a single error.



2. Development & Architecture

This chapter describes the structural organization of the project, the core architectural principles behind the engine, and the execution model that enables deterministic and reproducible simulation of swarm-based gameplay.

2.1 Project Hierarchy and Structural Organization

To ensure maintainability and extensibility, *Swarm Pastor* follows a clearly defined directory hierarchy that separates engine infrastructure, gameplay logic, data configuration, and world-level concerns. This structure enforces architectural boundaries and prevents tight coupling between unrelated components.

The project is organized as follows:

- **assets/**
Contains all visual and audio resources used by the game. Assets are treated as external data and are never directly coupled to gameplay logic.
- **core/**
Holds the foundational engine components, including the main application class, global settings, camera control, audio management, and the main game loop. This layer coordinates system execution and scene transitions.
- **data/**
Store databases corresponding with one .csv that save the information about the different matches
- **entities/**
Defines lightweight entity representations such as the player drone, swarm agents, predators, projectiles, and defensive elements. Entities store minimal state and do not implement complex behavior internally.
- **scenes/**
Manages the different high-level game states, including menus, tutorials, narrative sequences, and gameplay. Each scene encapsulates its own lifecycle, allowing clean transitions between modes.
- **systems/**
Contains the core behavioral logic of the game. Each system implements a specific concern—such as swarm behavior, predator AI, difficulty adjustment, or power management—by processing relevant entities every frame.

- **world/**
Handles spatial partitioning, chunk management, and environmental systems such as zones or beacons. This layer ensures scalability by limiting per-frame computation to relevant areas.
- **main.py**
Serves as the entry point of the application, initializing the engine and launching the initial scene.

This hierarchical organization allows complex behavior to arise from system interaction rather than from monolithic or deeply nested object logic.

2.2 The Engine Core: System-Oriented Design

2.2.1 Entity–System Architecture

In *Swarm Pastor*, entities are implemented as lightweight structures that store only state information, such as position, velocity, type, or energy levels. Entities do not contain decision-making logic or complex behavior.

All behavior is implemented through systems that operate over collections of entities each frame. For example, swarm behavior, predator decision-making, and difficulty adjustment are handled by separate systems that process relevant entities independently.

This design provides several advantages:

1. Decoupling: Systems do not require knowledge of each other's internal implementation.
2. Scalability: New behaviors can be introduced by adding new systems without modifying existing entities.
3. Emergence: Complex global behavior arises naturally from the interaction of simple systems rather than from scripted control flow.

By avoiding deep inheritance hierarchies, the architecture reduces rigidity and supports experimentation with emergent mechanics.

2.2.2 Deterministic Update Loop

Each frame, *Swarm Pastor* executes a deterministic update loop in which systems are processed in a fixed and predefined order. This ordering is critical for ensuring consistent behavior across frames, particularly in a real-time swarm simulation.

A simplified execution sequence is as follows:

1. Player input and drone control processing
2. Swarm behavior evaluation
3. Predator behavior and targeting
4. Difficulty and pacing adjustments
5. Movement and position integration
6. World and camera updates
7. Rendering

By maintaining a stable system order, the engine avoids race conditions and ensures that identical initial conditions produce identical outcomes. This determinism is essential for debugging emergent behavior and for maintaining predictable gameplay dynamics despite systemic complexity.

2.3 Swarm Behavior Engine

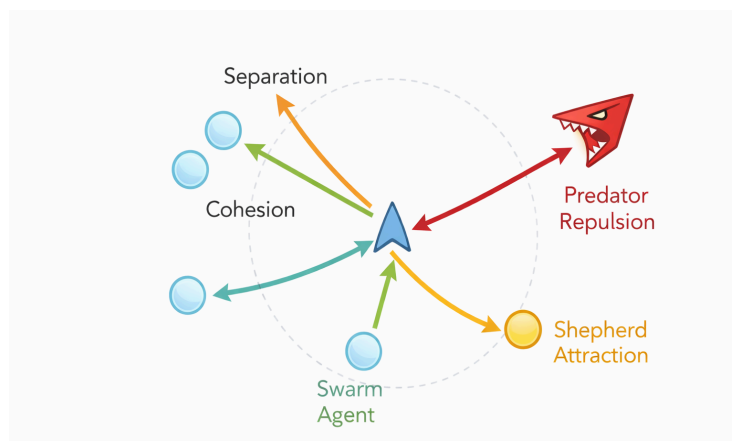


Figure 2.3 Local Forces to a swarm agent

The swarm behavior in *Swarm Pastor* is implemented as a fully decentralized system based on local interaction rules. Each swarm agent operates independently, with no global knowledge of the swarm state or the environment. Collective behavior emerges exclusively from the interaction of simple forces applied at the agent level.

2.3.1 Local Perception and Neighbor Sampling

Each swarm agent perceives its environment within a limited radius. Only nearby agents, predators, and the player drone are considered when computing behavior. This restriction is fundamental to the emergent nature of the system, as no agent has access to global information.

For each frame, a swarm agent samples:

- Nearby swarm agents (for cohesion and separation)
- The player drone (as a stabilizing reference)
- Nearby predators (as sources of threat)

By limiting perception to a local neighborhood, the system ensures scalability and prevents global coordination from being hard-coded into individual agents.

```
for other in self.boids:
    if other is b:
        continue
    d = other.pos - b.pos
    d2 = d.length_squared()
    if d2 <= nR2:
        center += other.pos
        avg_vel += other.vel
        count += 1
    if d2 <= sR2 and d2 > 1e-6:
        sep -= d.normalize() / max(1e-6, (d.length() / self.separation_radius))
```

Listing 2.3.1 Local neighbor sampling used by swarm agents. Each agent considers only nearby neighbors within a fixed perception radius, enabling emergent collective behavior without global coordination.

2.3.2 Force-Based Behavior Model

Swarm movement is governed by a force-based model in which multiple behavioral influences are combined into a single steering vector. These influences include:

- Cohesion: Attraction toward the local center of nearby agents
- Separation: Repulsion from agents that are too close
- Shepherd Attraction: Soft attraction toward the player drone
- Predator Repulsion: Strong avoidance force applied when threats are nearby

Each force is weighted independently, allowing fine-grained tuning of swarm behavior. The resulting steering vector is added to the agent's velocity, which is then clamped to a maximum magnitude to prevent instability.

This approach produces smooth and organic movement while remaining computationally lightweight.

```
cohesion_force = self._seek(b, center, max_speed)
alignment_force = avg_vel - b.vel
alignment_force = self._limit(alignment_force, self.max_force)
separation_force = self._limit(sep * max_speed, self.max_force)

force += cohesion_force * (self.w_cohesion * coh_m)
force += alignment_force * (self.w_alignment * ali_m)
force += separation_force * (self.w_separation * sep_m)

force += self._seek(b, anchor, max_speed) * (self.w_goal * goal_m)

if self.w_drone_avoid > 0.0:
    away = self._flee(b, drone_pos, max_speed)
    force += away * self.w_drone_avoid

if state == "PANIC" and threat_pos is not None and threat_recent:
    force += self._flee(b, threat_pos, max_speed) * 1.25

if jitter > 0.0:
    rnd = pygame.Vector2(random.uniform(-1, 1), random.uniform(-1, 1))
    force += rnd * (55.0 * jitter)

if b.pos.y < world_y_min + 10:
    force += pygame.Vector2(0, 140) * self.w_bounds
elif b.pos.y > world_y_max - 10:
    force += pygame.Vector2(0, -140) * self.w_bounds

b.vel += self._limit(force, self.max_force) * dt
b.vel = self._limit(b.vel, max_speed)
b.pos += b.vel * dt
```

Listing 2.3.2 Force-based steering model. Multiple behavioral influences are accumulated into a single steering vector; weighted independently, and clamped to ensure stable motion.

2.3.3 Mood-Driven Modulation and Panic States

In addition to local forces, swarm behavior is modulated by a global mood system that reflects the current stress level of the swarm. This mood is influenced by factors such as predator proximity, agent loss, and time spent under threat.

The system defines three high-level states:

- Calm: Stable movement, strong cohesion
- Alert: Increased separation and responsiveness
- Panic: High dispersion, erratic motion, reduced cohesion

Rather than directly controlling agents, the mood system adjusts force weights and noise parameters, indirectly influencing behavior. This preserves decentralization while enabling large-scale behavioral shifts under pressure.

```
if self.stress < 0.33:
    self.state = SwarmMood.CALM
elif self.stress < 0.66:
    self.state = SwarmMood.ALERT
else:
    self.state = SwarmMood.PANIC

def modifiers(self):
    """Return a dict of mood-based modifiers for the boids system."""

    s = self.stress

    # smooth-ish mapping
    cohesion_mul = 1.0 - 0.65 * s
    align_mul = 1.0 - 0.30 * s
    separation_mul = 1.0 + 1.10 * s
    goal_mul = 1.0 - 0.55 * s

    speed_mul = 1.0 + 0.45 * s
    jitter = 0.0 + 1.0 * s # for random steering

    return {
        "cohesion": cohesion_mul,
        "alignment": align_mul,
        "separation": separation_mul,
        "goal": goal_mul,
        "speed": speed_mul,
        "jitter": jitter,
        "state": self.state,
        "stress": self.stress,
        "threat_pos": self.last_threat_pos,
        "threat_strength": self.last_threat_strength,
        "threat_recent": self.threat_t < 1.0,
```

Listing 2.3.3 Swarm mood state selection and export of behavioral modifiers used to scale local agent forces under stress.

2.3.5 Emergent Intelligence Perspective

The swarm behavior implemented in *Swarm Pastor* exemplifies a form of emergent artificial intelligence. No individual agent exhibits complex reasoning or planning, yet the collective displays adaptive, goal-oriented behavior in response to environmental pressure and player influence.

Intelligence in this system does not reside in a single algorithm but emerges from:

- Local agent interactions
- Environmental constraints
- Predator-induced pressure
- Player-mediated stabilization

This aligns the project with broader research in artificial life and complex adaptive systems, demonstrating how simple rules can give rise to rich and unpredictable dynamics.

2.4 Enemy AI and Adaptive Difficulty Control

Enemy behavior in *Swarm Pastor* is designed to apply continuous, asymmetric pressure on the swarm rather than direct confrontation with the player. Enemies do not aim to defeat the player outright; instead, they destabilize the swarm, forcing the player to act as an indirect regulator.

2.4.1 Predator Behavior Models

Predators are implemented as autonomous agents with distinct behavioral profiles. Each predator type operates under a simple rule set, yet their interaction with the swarm produces complex outcomes.

The main predator archetypes are:

- **Grab Predators:**
Identify isolated swarm agents using local density heuristics and attempt to extract them from the group. Once a target is captured, the predator retreats, increasing swarm fragmentation.
- **Ranged Predators:**
Maintain distance from the swarm center and periodically fire projectiles toward dense regions, forcing dispersion without committing to direct contact.
- **Kamikaze Predators:**
Trigger area-based disruption when entering high-density zones, creating sudden local collapses in swarm cohesion.

Target selection is based on spatial heuristics such as distance to swarm centroid and neighbor count, avoiding explicit pathfinding or global planning.

```
def most_isolated_boid(self, boids):
    center = self.swarm_center(boids)
    best = None
    best_score = -1e9
    for b in boids:
        d = (b.pos - center).length()
        n = self.neighbor_count(b, boids)
        score = 1.2 * d + 80.0 * (1.0 / (n + 1))
        if score > best_score:
            best_score = score
            best = b
    return best

def densest_boid(self, boids):
    best = None
    best_n = -1
    for b in boids:
        n = self.neighbor_count(b, boids)
        if n > best_n:
            best_n = n
            best = b
    return best, best_n
```

Listing 2.4.1 Spatial heuristic targeting used by predators. The system selects vulnerable prey (isolated boids) and high-impact targets (dense regions) using swarm centroid distance and local neighbor counts, avoiding global planning or pathfinding.

2.4.2 Adaptive Difficulty Director

Rather than relying on fixed waves or scripted progression, *Swarm Pastor* employs a difficulty director that continuously adjusts enemy pressure based on player performance.

The director evaluates a normalized performance signal derived from:

- Current swarm size
- Recent agent losses
- Spatial cohesion of the swarm
- Time spent without major failures

This signal is used to dynamically regulate:

- Predator spawn frequency
- Maximum number of concurrent enemies
- Distribution of predator archetypes

As a result, difficulty adapts smoothly over time, increasing pressure when the player demonstrates stability and relaxing when the swarm approaches collapse.

```
# --- performance score P in [0,1] ---
close_score = min(1.0, boids_close / 14.0) # 14 close boids is "good"
streak_score = min(1.0, self.time_no_loss / 14.0) # faster ramp

# crowd penalty: only if it's REALLY crowded
crowd_penalty = 1.0
if predators_alive >= 11:
    crowd_penalty = 0.70
elif predators_alive >= 9:
    crowd_penalty = 0.82

# low boids => reduce difficulty a bit
alive_factor = 1.0
if boids_alive <= 6:
    alive_factor = 0.55
elif boids_alive <= 10:
    alive_factor = 0.78

P = (0.60 * close_score + 0.40 * streak_score) * crowd_penalty * alive_factor

# --- update D smoothly toward P ---
# rises steadily, falls faster (for forgiveness)
if P > self.D:
    self.D += (P - self.D) * min(1.0, 0.85 * dt)
else:
    self.D += (P - self.D) * min(1.0, 1.65 * dt)

self.D = max(0.05, min(0.98, self.D))
```

Listing 2.4.2: Adaptive difficulty director. A normalized performance score P is computed from swarm cohesion and stability indicators and is used to smoothly update the difficulty variable D via a feedback controller with asymmetric rise/fall rates

2.5 World Systems and Environmental Pressure

The world of *Swarm Pastor* is not a neutral container but an active component of the gameplay system. World-level mechanics are designed to introduce spatial constraints, pacing elements, and temporary environmental effects that interact with swarm behavior and enemy pressure.

Rather than relying on static levels, the game world is structured as a dynamic space that selectively applies pressure and opportunities, reinforcing the themes of adaptation and indirect control.

2.5.1 Spatial Partitioning and Chunk Management

To ensure scalability and performance, the game world is divided into spatial chunks that limit per-frame computation to relevant areas. Only entities within active chunks are updated, reducing unnecessary processing when the swarm moves across the map.

This approach allows the simulation to support a growing number of agents and environmental elements without compromising frame stability. Chunk-based partitioning also provides a foundation for future extensions, such as larger maps or region-based mechanics.

```
def update(self, scroll_x: float):
    current_idx = int(scroll_x // CHUNK_W)

    # generate chunks around camera
    for idx in range(current_idx - self.keep_behind, current_idx + self.keep_ahead + 1):
        if idx >= 0:
            self._ensure_chunk(idx)

    # cull far chunks
    to_del = []
    for idx in self.chunks.keys():
        if idx < current_idx - (self.keep_behind + 2) or idx > current_idx + (self.keep_ahead + 2):
            to_del.append(idx)
    for idx in to_del:
        del self.chunks[idx]

    # zones generation (based on chunk index)
    self.zones.generate_if_needed(current_idx)
```

Listing 2.5.1 *Chunk window management in an infinite horizontal world. The engine maintains only a limited range of chunks around the camera and culls distant chunks to ensure stable performance and bounded memory usage.*

2.5.2 Environmental Zones and Local Modifiers

The world includes temporary or persistent environmental zones that locally modify gameplay conditions. These zones apply effects such as increased danger, reduced visibility, or altered swarm behavior, encouraging the player to continuously reposition and adapt.

Zones are intentionally simple in implementation but powerful in effect. By altering parameters rather than introducing new rules, they integrate seamlessly with existing systems and preserve emergent behavior.

2.5.3 Beacons and Regrouping Mechanics

To counterbalance dispersal and fragmentation, the world periodically introduces beacons that act as temporary focal points. Beacons encourage swarm regrouping by providing a spatial objective that rewards cohesion.

These elements serve both mechanical and pacing purposes: they offer moments of recovery while also introducing time pressure, as failing to exploit a beacon opportunity can accelerate system collapse.

2.5.4 In-Game Objectives and Progression

Swarm Pastor includes a set of implicit in-game objectives that guide player improvement without relying on explicit missions. Progression is based on the player's ability to **maintain swarm cohesion**, **minimize agent loss**, and **recover quickly from disruptive events**.

As the player demonstrates stability, the adaptive difficulty system increases enemy pressure, exposing them to more complex scenarios. This creates a natural progression loop where improvement is measured through sustained control and resilience rather than through discrete rewards or scripted goals.

These objectives are embedded directly into the game mechanics, encouraging players to refine their understanding of swarm dynamics and indirect control over time.

3. Gameplay Design and Strategy

This chapter focuses on the gameplay implications of the systems described in Chapter 2. While *Swarm Pastor* is driven by artificial intelligence and emergent behavior, the player experience is shaped by how these systems are perceived, learned, and strategically exploited.

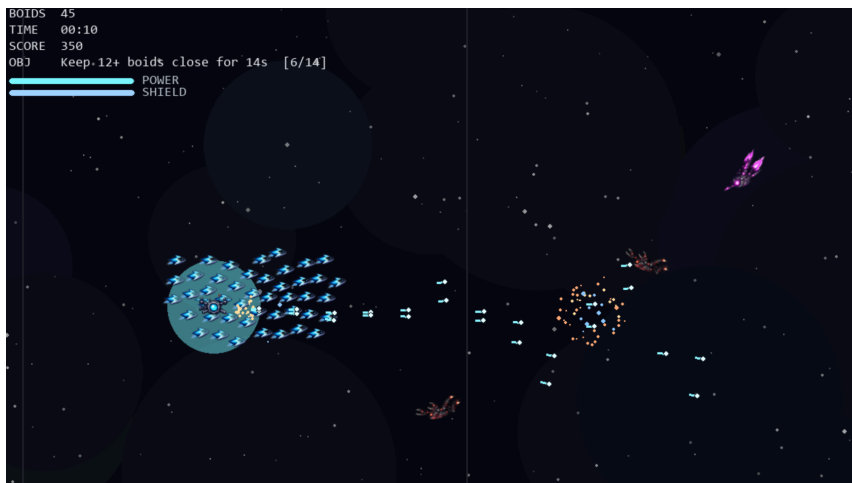
Rather than offering explicit objectives or scripted solutions, the game encourages players to develop intuition about swarm dynamics, spatial pressure, and risk management through repeated interaction with the system.

3.1 Indirect Control as a Core Gameplay Mechanic

Unlike traditional games where players directly command units or execute predefined actions, *Swarm Pastor* is built around indirect control. The player does not issue orders to the swarm; instead, they influence behavior through positioning, movement, and timing.

The player-controlled drone acts as a mobile reference point that modifies swarm behavior probabilistically rather than deterministically. This design choice forces the player to think in terms of tendencies and patterns rather than exact outcomes.

As a result, mastery of the game does not come from mechanical precision alone, but from understanding how the swarm reacts under different spatial configurations and threat levels.



3.2 Emergent Strategy and Player Learning

Strategies in *Swarm Pastor* are not explicitly taught or enforced. Instead, they emerge organically as the player observes the consequences of their actions.

Common strategic patterns include:

- Maintaining high swarm density to improve stability
- Prioritizing the protection of isolated agents
- Anticipating predator behavior rather than reacting to it
- Using environmental features, such as beacons or safe zones, to recover cohesion

Because the system is non-deterministic, these strategies are adaptive rather than fixed. Players are encouraged to continuously reassess the state of the swarm and adjust their behavior accordingly.

This learning process reinforces engagement and replayability, as no two sessions unfold in exactly the same way.

3.3 Risk Management and Decision-Making Under Pressure

A central challenge in *Swarm Pastor* is managing risk over time. The player is constantly faced with trade-offs between short-term stabilization and long-term survival.

For example, aggressively intercepting predators may temporarily reduce pressure but can lead to swarm dispersion if positioning is poor. Conversely, over-prioritizing cohesion may allow predators to exploit weaknesses at the swarm's periphery.

The absence of discrete win conditions shifts the focus from optimization toward survival. Success is measured by how long the player can delay systemic collapse rather than by completing a predefined objective.

3.4 Failure as a Systemic Process

Failure in *Swarm Pastor* is intentionally gradual rather than abrupt. The loss of individual agents weakens the swarm's ability to self-regulate, leading to increased instability and vulnerability.

This cascading failure model transforms mistakes into long-term consequences. Early errors may not immediately end the game, but they compound over time, increasing difficulty in subtle but persistent ways.

By framing failure as a systemic process, the game encourages reflection and learning rather than frustration.

3.5 Player Experience and Feedback

Despite the underlying complexity of the systems, the player interface is designed to provide clear and immediate feedback. Visual cues such as swarm dispersion, predator proximity, and environmental effects allow players to intuitively assess the state of the system without relying on explicit numerical indicators.

This emphasis on qualitative feedback supports the game's core philosophy: players should *feel* when the system is approaching instability rather than being explicitly told.

Replayability in *Swarm Pastor* emerges naturally from the interaction of adaptive difficulty, emergent behavior, and player learning. As players become more skilled, they are challenged not by scripted escalation but by increasingly subtle systemic pressures.

4. Accomplishments and Critical Evaluation

This chapter evaluates the results of the project from both a technical and a design perspective. Rather than focusing solely on achievements, it critically examines the strengths and limitations of the implemented solution, highlighting lessons learned during development.

4.1 Gameplay and Design Achievements

From a gameplay perspective, *Swarm Pastor* succeeds in translating swarm intelligence into a meaningful interactive challenge. Indirect control and emergent failure create a distinctive player experience that differs from traditional action or strategy games.

The gradual, systemic nature of failure encourages learning and adaptation rather than trial-and-error optimization. Players are rewarded for understanding patterns and dynamics instead of memorizing fixed strategies.

Additionally, the integration of adaptive difficulty ensures that pressure remains context-sensitive, allowing both novice and experienced players to engage with the system at an appropriate challenge level.

4.2 Limitations and Challenges

Despite its strengths, the project presents several limitations. Emergent systems are inherently difficult to balance, and small parameter changes can produce disproportionately large effects on gameplay difficulty and stability.

Debugging also posed a significant challenge. Because behavior emerges from system interaction rather than explicit control flow, identifying the root cause of undesirable outcomes often required extensive observation and experimentation.

Furthermore, the minimalist scope of the project means that certain elements—such as long-term progression, narrative depth, or advanced enemy coordination—remain underdeveloped. These limitations were accepted trade-offs in order to maintain focus on the core research objectives.

4.3 Lessons Learned

A key lesson from this project is that simplicity at the agent level can lead to significant complexity at the system level. Designing for emergence requires careful architectural planning and a willingness to embrace unpredictability.

The project also highlighted the importance of visual and qualitative feedback in helping players understand complex systems. Clear feedback mechanisms proved more effective than explicit numerical indicators in conveying system state.

4.4 Future Work

Several extensions could further enhance *Swarm Pastor*. These include more diverse environmental effects, additional predator archetypes with cooperative behavior, and deeper progression systems that preserve the game's emergent nature.

From a research perspective, the project could be expanded to explore learning-based adaptations, such as predators that evolve strategies based on player behavior, while maintaining the core principles of indirect control.

5. Bibliography

[1] Reynolds, C. W. (1987). *Flocks, herds, and schools: A distributed behavioral model*. ACM SIGGRAPH Computer Graphics, 21(4), 25–34.

[2] Schell, J. (2008). *The Art of Game Design: A Book of Lenses*. Morgan Kaufmann.

[3] Dormans, J. (2012). *Engineering Emergent Gameplay*. Proceedings of the Foundations of Digital Games Conference.

[4] Pygame Community. *Pygame Documentation*.